

دانشگاه تهران

پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر

تمرین شماره: ۴

شبکه‌های عصبی و یادگیری عمیق

نام و نام خانوادگی: آرتین توسلی

شماره دانشجویی: ۸۱۰۱۰۲۵۴۳

نیم‌سال دوم

سال تحصیلی ۴۰۴-۴۰۵

فهرست مطالب

۳	Sentiment Analysis Using Vanilla RNN/LSTM/GRU	۱
۳	تئوری	۱.۱
۳	Backpropagation Through Time	۱.۱.۱
۶	معماری دروازه ای	۲.۱.۱
۱۵	پیاده سازی	۲.۱
۱۵	پیش پردازش داده ها و واژه نامه	۱.۲.۱
۱۸	معماری مدل و بارگذاری داده	۲.۲.۱
۲۰	منحنی های یادگیری و مقایسه عملکرد	۳.۲.۱
۲۱	تشخیص رفتار گرادین نسبت به گام های زمانی	۴.۲.۱

فهرست تصاویر

۲۱	 روند تغییرات خطا و دقت آموزش و اعتبارسنجی مدل‌ها	۱۰۱
۲۲	 نمودار لگاریتمی جریان میانگین نرم‌گرادیان‌ها در طول گام‌های زمانی	۲۰۱

سوال ۱

Sentiment Analysis Using Vanilla RNN/LSTM/GRU

۱.۱ تئوری

۱.۱.۱ Backpropagation Through Time

الف) زنجیره گرادیان:

$$\frac{\partial L_T}{\partial W_{hh}} = \frac{\partial L_T}{\partial y^{pred}} \frac{\partial y^{pred}}{\partial h_T} \left[\frac{\partial h_T}{\partial W_{hh}} + \frac{\partial h_T}{\partial h_{T-1}} \frac{\partial h_{T-1}}{\partial W_{hh}} + \dots + \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial W_{hh}} \right] = \frac{\partial L_T}{\partial y^{pred}} \frac{\partial y^{pred}}{\partial h_T} \left[\frac{\partial h_T}{\partial W_{hh}} + \sum_{j=1}^T \frac{\partial h_j}{\partial h_{j-1}} \frac{\partial h_{j-1}}{\partial W_{hh}} \right]$$

فرض می‌کنیم تابع L_T با یک فرمولی مانند cross entropy مرتبط است (هر تابع اسکالر دیگری هم مشکلی ندارد و ما اینجا مشتقش را نخواهیم گرفت).

$$\frac{\partial L_T}{\partial W_{hh}} = \frac{\partial L_T}{\partial y^{pred}} \frac{\partial y^{pred}}{\partial h_T} \frac{dh_T}{dW_{hh}}$$

حال چون به صورت W_{hh} مستقیم و غیرمستقیم استفاده می‌شود، بخاطر همین مشتق آن نیز دو بخش خواهد داشت.

$$\frac{dh_T}{dW_{hh}} = \left(\frac{\partial h_T}{\partial W_{hh}} \right)_{h_{T-1}} + \frac{\partial h_T}{\partial h_{T-1}} \frac{dh_{T-1}}{dW_{hh}}$$

در فرمول بالا بجای T می‌آییم $T-1$ جایگذاری میکنیم:

$$\frac{dh_{T-1}}{dW_{hh}} = \left(\frac{\partial h_{T-1}}{\partial W_{hh}} \right)_{h_{T-2}} + \frac{\partial h_{T-1}}{\partial h_{T-2}} \frac{dh_{T-2}}{dW_{hh}}$$

حال با جایگذاری یک مرحله فرمول بازگشتی امون را برگشتیم:

$$\frac{dh_T}{dW_{hh}} = \left(\frac{\partial h_T}{\partial W_{hh}} \right)_{h_{T-1}} + \frac{\partial h_T}{\partial h_{T-1}} \left[\left(\frac{\partial h_{T-1}}{\partial W_{hh}} \right)_{h_{T-2}} + \frac{\partial h_{T-1}}{\partial h_{T-2}} \frac{dh_{T-2}}{dW_{hh}} \right]$$

$$\frac{dh_T}{dW_{hh}} = \left(\frac{\partial h_T}{\partial W_{hh}} \right)_{h_{T-1}} + \frac{\partial h_T}{\partial h_{T-1}} \left(\frac{\partial h_{T-1}}{\partial W_{hh}} \right)_{h_{T-2}} + \frac{\partial h_T}{\partial h_{T-1}} \frac{\partial h_{T-1}}{\partial h_{T-2}} \frac{dh_{T-2}}{dW_{hh}}$$

اگر به همین صورت مراحل بازگشتی را برگردیم، در نهایت خواهیم داشت:

$$\frac{dh_T}{dW_{hh}} = \left(\frac{\partial h_T}{\partial W_{hh}} \right)_{h_{T-1}} + \dots + \left(\frac{\partial h_T}{\partial h_{T-1}} \frac{\partial h_{T-1}}{\partial h_{T-2}} \dots \frac{\partial h_2}{\partial h_1} \right) \left(\frac{\partial h_1}{\partial W_{hh}} \right)_{h_0}$$

ساده تر:

$$\frac{dh_T}{dW_{hh}} = \sum_{k=1}^T \left(\prod_{j=k+1}^T \frac{\partial h_j}{\partial h_{j-1}} \right) \left(\frac{\partial h_k}{\partial W_{hh}} \right)_{h_{k-1}}$$

یعنی در نهایت خواهیم داشت:

$$\frac{\partial L_T}{\partial W_{hh}} = \frac{\partial L_T}{\partial h_T} \frac{dh_T}{dW_{hh}}$$

$$\frac{\partial L_T}{\partial W_{hh}} = \frac{\partial L_T}{\partial y^{pred}} \frac{\partial y^{pred}}{\partial h_T} \sum_{k=1}^T \left(\prod_{j=k+1}^T \frac{\partial h_j}{\partial h_{j-1}} \right) \left(\frac{\partial h_k}{\partial W_{hh}} \right)_{h_{k-1}}$$

(ب) از ما خواسته شده است که اثبات کنیم سقف نرم طیفی $J_{j-1} = \frac{\partial h_j}{\partial h_{j-1}}$ ، نرم طیفی W_{hh} می باشد.

یا به عبارتی،

$$\|J_{j-1}\|_2 \leq \|W_{hh}\|_2$$

در ابتدا از فرمول زیر مشتق گرفته تا ژاکوبین را محاسبه کنیم (فرض کنید داخل $\tanh$ را با z_j نمایش بدهیم).

$$h_j = \tanh(W_{hh}h_{j-1} + W_{xh}x_j + b_h)$$

$$J_{j-1} = \frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \tanh(z_j)}{\partial z_j} \times \frac{\partial z_j}{\partial h_{j-1}} = \text{diag}(1 - \tanh^2(z_j))W_{hh}$$

حال برای اینکه به حکم سوال نزدیک بشویم، از دو طرف تساوی نرم طیفی میگیریم.

$$\|J_{j-1}\|_2 = \|\text{diag}(1 - \tanh^2(z_j))W_{hh}\|_2$$

با توجه به خاصیت Sub-multiplicativity ($\|AB\|_2 \leq \|A\|_2 \|B\|_2$):

$$\|J_{j-1}\|_2 \leq \|\text{diag}(1 - \tanh^2(z_j))\|_2 \times \|W_{hh}\|_2$$

بیشترین کشیدگی در ماتریس قطری برابر با بزرگترین مقدار قدر مطلق روی قطر می‌باشد ($\|\text{diag}(d_1, \dots, d_d)\|_2 = \max_i |d_i|$) و چون $\tanh$ بین -1 و 1 می‌باشد یعنی $1 - \tanh^2(z_j)$ بین 0 و 1 می‌باشد و یعنی مقادیر روی قطر این ماتریس حداکثر 1 می‌باشد و یعنی $\|\text{diag}(1 - \tanh^2(z_j))\|_2$ حداکثر 1 می‌باشد پس داریم:

$$\|J_{j-1}\|_2 \leq 1 \times \|W_{hh}\|_2$$

حال میبایست با کمک این فرمول vanishing/exploding gradient را توضیح بدیم. در آخر قسمت الف به جواب زیر رسیدیم:

$$\frac{\partial L_T}{\partial W_{hh}} = \frac{\partial L_T}{\partial y^{pred}} \frac{\partial y^{pred}}{\partial h_T} \sum_{k=1}^T \left(\prod_{j=k+1}^T \frac{\partial h_j}{\partial h_{j-1}} \right) \left(\frac{\partial h_k}{\partial W_{hh}} \right)_{h_{k-1}}$$

ما می‌خواهیم از دو طرف تساوی بالا نرم 2 اقلیدسی بگیریم. توجه کنید که نرم 2 اگر بجای بردار، روی ماتریس اعمال شود همان نرم طیفی میشود، طبق تعریف زیر:

$$\|A\|_2 = \max_{v \neq 0} \frac{\|Av\|_2}{\|v\|_2}$$

$$\|A\|_2 \geq \frac{\|Av\|_2}{\|v\|_2}$$

$$\|A\|_2 \|v\|_2 \geq \|Av\|_2$$

حال فرض کنید

$$w = \|W_{hh}\|_2.$$

طبق اثبات قسمت پیش داریم:

$$\left\| \prod_{j=k+1}^T J_{j-1} \right\|_2 \leq \prod_{j=k+1}^T \|J_{j-1}\|_2 \leq w^{T-k}$$

بنابراین، سقف اندازه گرادیان رابطه مستقیم با $\sum_{k=1}^T w^{T-k}$ دارد. حال اگر مقدار $w = \|W_{hh}\|_2$ کوچکتر از 1 باشد آن موقع در $T \rightarrow \infty$ ، vanishing gradient رخ می‌دهد چون

$$\lim_{(T-k) \rightarrow \infty} w^{T-k} = 0$$

یعنی سقف گرادیان 0 می‌شود پس خودش هم 0 می‌شود. نکته اینجاست که اگر w بزرگتر از 1 باشد این حد به بینهایت میل می‌کند ولی چون این سقف اندازه گرادیان می‌باشد نمیتوان لزوماً گفت exploding gradient رخ داد.

میبایست فرضی دیگر کرد تا به exploding gradient برسیم، در واقع باید شرایطی را در نظر بگیریم که نرم ژاکوبین برابر $w = \|W_{hh}\|_2$ بشود آن موقع T بار که ژاکوبین ها ضرب میشوند به معنای خود اندازه گرادیان (نه سقف آن) رابطه مستقیم با $\sum_{k=1}^T w^{T-k}$ دارد و اگر w بزرگتر از 1 باشد آن موقع در

$\infty, T \rightarrow \infty$ exploding gradient رخ میدهد طبق رابطه زیر:

$$\lim_{(T-k) \rightarrow \infty} w^{T-k} = \infty$$

حال چطور چنین شرایطی برقرار کنیم، با توجه به رابطه زیر که بالاتر به آن رسیده بودیم اگر بردار z_j صفر باشد (یعنی همه نورون های مخفی تقریباً مقدار صفر داشته باشند) آن موقع $\tanh$ اش صفر می شود و $\text{diag}(1 - \tanh^2(z_j))$ همان ماتریس همانی میشود پس یعنی:

$$\|J_{j-1}\|_2 = \|\text{diag}(1 - \tanh^2(z_j))W_{hh}\|_2$$

$$\|J_{j-1}\|_2 = \|W_{hh}\|_2$$

حال طبق همان توضیح بالاتر به exploding gradient میرسیم

ج) در این روش ما به جهت گرادیان g دست نمی زنیم اما اگر اندازه آن از آستانه τ که تعریف کردیم بیشتر بشود scale down اش میکنیم. (اگر زیر آستانه بود، کاری نمیکنیم). در این حالت:

$$g_{new} = \frac{\tau}{\|g\|_2} g$$

در این صورت گرادییانی که وزن ها را میخواهد آپدیت کند مقدار بسیار زیادی پیدا نخواهد کرد و آپدیت های وزن ها نویزی نمی شود و یعنی آموزش پایدار تر نیز می شود. با اینکار چون اندازه گرادیان را محدود کردیم دیگر ممکن نیست مقدار بسیار زیاد پیدا کند ولی همینطور که در توضیح روش گفته شد برای مقدار گرادیان کم (نزدیک ۰) چون کمتر از آستانه هست الگوریتم دستی به گرادیان نمی زند. اگر قرار بود در اون حالت scale up کنیم، زمانی که آموزش به یک local minimum خوب میرسید ما چون نمیگذاریم گرادیان ۰ بشود، وزن ها هنوز آپدیت میشوند و از این جای خوب درمیاییم یعنی عملاً شبیه نویز اضافه کردن هست کف تعریف کردن برای گرادیان (که وقتی کمتر از اون کف میشود، نویز اضافه میشود که شبکه نیاز باشد باز آموزش ببیند). مشکلی مهمتر، حتی اگر نقطه پایان مسئله مون را کنار بگذاریم (convergence)، اساساً گرادیان کوچک و بزرگ تفاوت مهمی دارند و این هست که گرادیان کوچک بسیار نویزی میباشد و عملاً جهت خود را گم کرده است و اگر ما scale up کنیم عملاً با یک گرادیان تقریباً نویزی حرکت میکنیم که ایده بسیار بدی است ولی گرادیان بزرگ جهت اش درست هست و ما فقط به اندازه آن داریم دست میزنیم پس ما را گمراه نمیکند.

۲.۱.۱ معماری دروازه ای

الف) معادلات LSTM: آپدیت شدن حافظه بلند مدت:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

ضریب نگهداری حافظه جدید:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

ضریب نگهداری حافظه بلند مدت:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

حافظه جدید:

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

آپدیت شدن حافظه کوتاه مدت:

$$h_t = \sigma(W_h \cdot [h_{t-1}, x_t] + b_h) \odot \tanh(c_t)$$

حال از رابطه اول نسبت به c_{t-1} مشتق میگیریم:

$$\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t) + \text{diag}(c_{t-1}) \frac{\partial f_t}{\partial c_{t-1}} + \text{diag}(i_t) \frac{\partial \tilde{c}_t}{\partial c_{t-1}} + \text{diag}(\tilde{c}_t) \frac{\partial i_t}{\partial c_{t-1}}$$

چون $f_t, i_t, \tilde{c}_t$ به h_{t-1} وابسته اند و خود h_{t-1} طبق فرمول زیر به c_{t-1} وابسته است، ما یک وابستگی غیرمستقیم داریم:

$$h_{t-1} = \sigma(W_h \cdot [h_{t-2}, x_{t-1}] + b_h) \odot \tanh(c_{t-1})$$

برای راحتی، عبارت $\sigma(W_h \cdot [h_{t-2}, x_{t-1}] + b_h)$ را o_{t-1} در نظر میگیریم. پس:

$$h_{t-1} = o_{t-1} \odot \tanh(c_{t-1})$$

$$\frac{\partial h_{t-1}}{\partial c_{t-1}} = \text{diag}(o_{t-1}) \text{diag}(1 - \tanh^2(c_{t-1}))$$

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

میتوان W_f را به دو بخش شکوند:

$$z_t = W_f \cdot [h_{t-1}, x_t] + b_f = W_{fh} h_{t-1} + W_{fx} x_t + b_f$$

$$f_t = \sigma(z_t)$$

$$\frac{\partial f_t}{\partial h_{t-1}} = \frac{\partial f_t}{\partial z_t} \times \frac{\partial z_t}{\partial h_{t-1}}$$

$$\frac{\partial f_t}{\partial z_t} = \text{diag}(\sigma(z_t) \odot (1 - \sigma(z_t)))$$

و چون f_t همان $\sigma(z_t)$ است:

$$\frac{\partial f_t}{\partial z_t} = \text{diag}(f_t \odot (1 - f_t))$$

$$\frac{\partial z_t}{\partial h_{t-1}} = W_{fh} \frac{\partial f_t}{\partial h_{t-1}} = \text{diag}(f_t \odot (1 - f_t)) W_{fh}$$

به طور مشابه:

$$\frac{\partial i_t}{\partial h_{t-1}} = \text{diag}(i_t \odot (1 - i_t)) W_{ih}$$

بسیار مانند بالا صرفا بجای سیگموید $\tanh$ استفاده شده است که مشتق آن $1 - \tanh^2$ هست پس یعنی:

$$\frac{\partial \tilde{c}_t}{\partial h_{t-1}} = \text{diag}(1 - \tilde{c}_t^2) W_{ch}$$

پس در نهایت جایگذاری میکنیم از $\text{diag}(a) \times \text{diag}(b) = \text{diag}(a \odot b)$ استفاده کردیم برای کمی ساده سازی):

$$\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t) + \left[\text{diag}(c_{t-1}) \frac{\partial f_t}{\partial h_{t-1}} + \text{diag}(i_t) \frac{\partial \tilde{c}_t}{\partial h_{t-1}} + \text{diag}(\tilde{c}_t) \frac{\partial i_t}{\partial h_{t-1}} \right] \frac{\partial h_{t-1}}{\partial c_{t-1}}$$

تبدیل میشود به:

$$\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t) + \left[\text{diag}(c_{t-1} \odot f_t \odot (1 - f_t)) W_{fh} + \text{diag}(i_t \odot (1 - \tilde{c}_t^2)) W_{ch} + \text{diag}(\tilde{c}_t \odot i_t \odot (1 - i_t)) W_{ih} \right] \times \text{diag}(o_{t-1} \odot (1 - \tanh^2(c_{t-1})))$$

در واقعیت عبارت داخل براکت بسیار کوچک میشود چون در همه عبارت ها مشتق توابع فعال ساز ضرب شده است که برای سیگمویید حداکثر 0.25 هست و برای $\tanh$ حداکثر ۱ هست. میشود تقریب زد و گفت

$$\frac{\partial c_t}{\partial c_{t-1}} \approx \text{diag}(f_t)$$

حال vanishing gradient وقتی رخ میدهد که تاریخچه بلندمدتش را فراموش میکند، در اینجا اما نباید فرض کنیم forget نمیکند یعنی f_t تقریباً ۱ باشد آن موقع تقریباً میتوان گفت:

$$\frac{\partial c_t}{\partial c_{t-1}} \approx I$$

حال بیایم backpropagate را ببینیم:

$$\frac{\partial L_T}{\partial c_k} = \frac{\partial L_T}{\partial c_T} \frac{\partial c_T}{\partial c_k} = \frac{\partial L_T}{\partial c_T} \prod_{j=k+1}^T \frac{\partial c_j}{\partial c_{j-1}}$$

برعکس شبکه vanilla که در سوال پیش نشون دادیم بخاطر W_{hh}^{T-k} مشکل vanishing gradient داشتیم اینجا با فرض f_t تقریباً ۱ مشاهده میکنیم که ضرب یک سری I شده و یعنی اندازه گرادیان افقی نمیکند.

توجه کنید که شبکه f_t را تعیین میکند و یعنی خود شبکه تصمیم میگیرد که ارتباط های دورش را در نظر بگیرد یا نه (به راحتی اگر این تصمیم را داشت، f_t را مثلاً 0.5 میتواند بگذارد و باز vanishing gradient رخ میدهد ولی نکته اینجاست که این تصمیم باید در دست شبکه باشد، برعکس vanilla که شبکه همواره ناچار به فراموش بود.)

(ب) برای تبدیل LSTM به GRU در ابتدا باید معادلات هر دو را نگاه بکنیم:

معادلات LSTM: آپدیت شدن حافظه بلند مدت:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

ضریب نگهداری حافظه جدید:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

ضریب نگهداری حافظه بلند مدت:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

حافظه جدید:

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

آپدیت شدن حافظه کوتاه مدت:

$$h_t = \sigma(W_h \cdot [h_{t-1}, x_t] + b_h) \odot \tanh(c_t)$$

معادلات GRU: گیت آپدیت:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

گیت ریست:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

حافظه جدید:

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$$

آپدیت حافظه:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

سعی میکنیم فرمول آپدیت شدن حافظه بلند مدت در LSTM را به فرمول آپدیت حافظه در GRU تبدیل کنیم (توجه کنید در GRU ما دیگر c_t نداریم چون دو حافظه مجزا نگه نمیدارد). در ابتدا میبینیم ضرایب فرمول آپدیت حافظه GRU مکمل هم هستند پس همین فرض را برای LSTM نیز میکنیم

$$f_t + i_t = 1 \implies f_t = 1 - i_t$$

توی LSTM چون هر دوی اینا جدا بودن، ممکن بود هر دو صفر باشند و یعنی نه چیزی از قبل نگه دار و نه چیزی یاد بگیر، در GRU ولی دقیقا مکمل هم تعریف میکند این دو را.

تغییرات LSTM بعد از این فرض:

$$c_t = (1 - i_t) \odot c_{t-1} + i_t \odot \tilde{c}_t$$

مقایسه با GRU:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

حال که ضرایب را درست کردیم، باید کلا یک حافظه نگه داریم بخاطر همین h_t را از LSTM حذف میکنیم یعنی در معادله زیر که برای LSTM بود:

$$h_t = \sigma(W_h \cdot [h_{t-1}, x_t] + b_h) \odot \tanh(c_t)$$

دروازه خروجی $o_t = W_h \cdot [h_{t-1}, x_t] + b_h$ از سیگموید عبور داده میشود و بعد به معنای این هست که حافظه بلند مدت آپدیت شده چه مقداری ازش را میخواید در این لحظه (توجه کنید h_t را ما در لحظه t خروجی میدهیم). ما میخواهیم این حافظه را حذف کنیم پس دروازه خروجی را ۱ قرار میدهیم و یعنی خروجی لحظه ای رو میگذاریم هر چی تا الان بوده خروجی داده شود و با ۱ گذاشتن عملا h_t و c_t را یکی کردیم. توجه کنید از $\tanh$ صرف نظر کردیم چون اگر

نمی‌کردیم:

$$h_t = \tanh(c_t)$$

$$c_t = (1 - i_t) \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = \tanh((1 - i_t) \odot c_{t-1} + i_t \odot \tilde{c}_t)$$

اول اینکه اصلاً نیازی برای $\tanh$ نیست چون خود $\tilde{c}_t$ یکبار از $\tanh$ رد شده بود و خروجیش بین -1 و 1 محدود شده بود و یعنی c_t نیز خروجیش در این بازه محدود شده است و اگر ما بیخود لایه های $\tanh$ بگذاریم ممکن است به مشکل vanishing gradient بخوریم بخاطر همین به راحتی فرض می‌کنیم $h_t = c_t$ می‌باشد. معادلات LSTM بعد از اعمال فرضای بالا:

$$h_t = (1 - i_t) \odot h_{t-1} + i_t \odot \tilde{c}_t$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

حال قدم نهایی برای این تبدیل این است که فرض کنیم گیت ریست در GRU یک می‌باشد (چون چنین چیزی در LSTM وجود ندارد و ما فرض می‌کنیم کل حافظه قبلی را عبور دهد). معادلات GRU بعد از اعمال این فرض:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

$$\tilde{h}_t = \tanh(W_h \cdot [h_{t-1}, x_t] + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

و دقیقاً با LSTM بالا یکی شد.

مزیت GRU:

در شبکه LSTM ما برای محاسبه ۴ متغیر $(i_t, f_t, o_t, \tilde{c}_t)$ نیاز به ۴ مجموعه ماتریس وزن و بایاس مستقل داریم. اما در GRU تنها به ۳ مجموعه وزن $(z_t, r_t, \tilde{h}_t)$ نیاز است یعنی GRU سریع‌تر آموزش می‌بیند و از نظر memory هم به صرفه تر می‌باشد. همچنین پارامتر بیشتر یعنی مدل پیچیده تر هست و احتمال overfit شدنش بیشتر است و یعنی GRU از این لحاظ مقاوم تر هست.

مزیت LSTM:

در شبکه LSTM، به دلیل مجزا بودن حافظه داخلی (c_t) از خروجی (h_t) و وجود دروازه خروجی (o_t) ، شبکه می‌تواند اطلاعات مهمی را در حافظه خودش ذخیره کند اما تصمیم بگیرد آن را در همون لحظه بیرون ندهد. شبکه می‌تواند این اطلاعات را با استفاده از o_t تا زمانی که واقعاً به آن‌ها نیاز است نگه میدارد. در معماری GRU، چون c_t و h_t با هم یکی شده‌اند، هرگونه به‌روزرسانی در حافظه، بلافاصله و کاملاً در خروجی (h_t) دیده می‌شود و به بعدی

هاش پاس داده میشود. هر اطلاعاتی که در GRU ذخیره می‌شود، بلافاصله روی لایه‌های بعدی و پیش‌بینی‌های لحظه‌ای تأثیر می‌گذارد. در شبکه‌های پیچیده و دنباله‌های بسیار طولانی، این ناتوانی در پنهان کردن، باعث می‌شود GRU نتواند وابستگی‌های تودرتو و دور از هم را به خوبی LSTM مدل‌سازی کند. یعنی عملاً وقتی رابطه کوتاه مدت و بلند مدت را یکجا نگه میداریم ذاتاً با هم در مشکل دارند و یک tradeoff ای دارند، اگر رابطه بلند مدت بسیار پیچیده بشود چون بخشی از تمرکز حافظه روی کوتاه مدت گذاشته شده است قدرت تحلیل درست از اون رابطه بلند را ندارد (چه بسا اون تمرکز ممکن است گمراه نیز بکند رابطه بلند را مثل اینکه ضمیر متصلی که می‌خواهیم ببینیم به کجا مربوط است و اگر خیلی دور از ضمیرش نام برده باشیم و یک ضمیر نامربوط دیگر هم نزدیکتر استفاده کرده باشیم، قدرت اینکه اون دورتری را تشخیص بدهد از دست میدهد و اون نزدیک تری گمراهش نیز میکند و به اشتباه ضمیر نزدیک تر را میگوید به این ضمیر متصل ربط دارد).

ج) در ابتدا متغیرهای زیر را تعریف میکنیم: x_t : بیت ورودی در زمان t : آیا تعداد ۱ها فرد است؟ (۱ بیت) منطق آپدیت شدن:

$$c_t = \text{XOR}(x_t, c_{t-1}) = [x_t \wedge \sim c_{t-1}] \vee [\sim x_t \wedge c_{t-1}]$$

حال باید این فرمول را به فرمول آپدیت شدن حافظه بلند مدت LSTM تبدیل کنیم. غیرممکن است همزمان $\sim x_t \wedge c_{t-1}$ و $x_t \wedge \sim c_{t-1}$ باشند پس با خیال راحت جای OR بین این عبارت + می‌گذاریم (توجه کنید که همه چیز ۱ بیت هست).

$$c_t = [x_t \wedge \sim c_{t-1}] + [\sim x_t \wedge c_{t-1}]$$

حال پس باید تناظرهای زیر را برقرار کنیم:

حالت $[\sim x_t \wedge c_{t-1}]$ با $f_t \odot c_{t-1}$: با نوشتن جدول درستی متوجه می‌شویم $\odot$ همان And است وقتی تک بیت هستیم.

$$f_t \wedge c_{t-1} \longleftrightarrow \sim x_t \wedge c_{t-1}$$

$$f_t = \sigma(W_{fh}h_{t-1} + W_{xh}x_t + b_f)$$

$$h_{t-1} = o_{t-1} \odot \tanh(c_{t-1})$$

$$\implies h_{t-1} = o_{t-1} \wedge c_{t-1}$$

$$\implies h_{t-1} = c_{t-1}$$

$$\implies f_t = \sigma(W_{fh}c_{t-1} + W_{xh}x_t + b_f)$$

متغیر c_{t-1} یا صفر است یا یک پس طبق تعریف $\tanh$ سوال می‌توان به ازای این مقادیر، $c_{t-1} = \tanh(c_{t-1})$ گفت. o_{t-1} هم چون از سیگموید می‌گذرد طبق تعریف سوال، خروجی باینری می‌دهد پس $\odot$ را می‌توان با And جایگزین کرد. پارامترهای o_{t-1} را جوری تنظیم می‌کنیم که ۱ بشود. با تنظیم پارامترها کاری می‌کنیم به صورت منطقی $f_t = \sim x_t$ بشود.

$$W_{fh} = 0, W_{xh} = -2, b_f = 1$$

$$o_t = \sigma(W_{ho}h_{t-1} + W_{xo}x_t + b_h)$$

گفتیم که می‌خواهیم o_t همیشه ۱ باشد، بسیار راحت می‌توان فارغ از پارامترهای h_{t-1} و x_t مقدار درون سیگموئید را بزرگتر از صفر کرد که همیشه ۱ بدهد.

$$b_h = 1 \text{ و } W_{ho} = W_{xo} = 0$$

حالت $[x_t \wedge \sim c_{t-1}]$ با $i_t \odot \tilde{c}_t$: بسیار مشابه روند قبل عمل می‌کنیم.

$\tilde{c}_t$	i_t	$x_t \wedge \sim c_{t-1} = i_t \wedge \tilde{c}_t$	$\sim c_{t-1}$	x_t
•	•	•	•	•
•	•	•	۱	•
۱	•	•	•	۱
۱	۱	۱	۱	۱

توجه کنید $\tilde{c}_t$ ، i_t را در اینجا یک حالت بهش دادیم و بعد با تنظیم پارامتر مجبورش میکنیم همان بشود (حالت‌های دیگر نیز ممکن بود) معادله هر سطر برای ستون‌های i_t و $\tilde{c}_t$ نوشته شد و بعد یکی از حالت‌هایی که این معادلات به ازای آن برقرار بودند انتخاب کردم.

$$\tilde{c}_t = \tanh(W_{hc}h_{t-1} + W_{xc}x_t + b_c)$$

می‌دانیم : $h_{t-1} = c_{t-1}$

سطر ۱ : $W_{hc} + b_c = 0$

سطر ۲ : $b_c = 0$

سطر ۳ : $W_{hc} + W_{xc} + b_c > 0$

سطر ۴ : $W_{xc} + b_c > 0$

$$\implies b_c = 0 \text{ و } W_{xc} = 1 \text{ و } W_{hc} = 0$$

$$i_t = \sigma(W_{hi}h_{t-1} + W_{xi}x_t + b_i)$$

سطر ۱ : $W_{hi} + b_i \leq 0$

سطر ۲ : $b_i \leq 0$

سطر ۳ : $W_{hi} + W_{xi} + b_i \leq 0$

سطر ۴ : $W_{xi} + b_i > 0$

$$\implies b_i = -1, W_{hi} = -4, W_{xi} = 3$$

در لحظه $t = 0$ ، هیچ بیتی پردازش نشده است و تعداد یک‌های مشاهده‌شده برابر با صفر است و یعنی زوج است پس:

$$c_0 = 0$$

همچنین با توجه به طراحی ما برای باز بودن همیشگی دروازه خروجی ($o_t = 1$) و رابطه $h_t = o_t \odot \tanh(c_t)$ ، حالت پنهان اولیه نیز به طور طبیعی برابر با صفر خواهد بود:

$$h_0 = 0$$

اثبات صحت طراحی

ثابت می‌کنیم برای هر گام زمانی $t \geq 0$ ، مقدار درون حالت سلول (c_t) دقیقاً برابر با مقدار زوجیت دنباله ورودی (p_t) تا آن لحظه است ($c_t = p_t$).

پایه: برای $t = 0$ ، هیچ ورودی دریافت نشده است. تعداد یک‌ها ۰ و در نتیجه $p_0 = 0$ است. با توجه به مقداردهی اولیه شبکه، داریم $c_0 = 0$. پس $c_0 = p_0$ و حکم برای گام پایه برقرار است.

فرض استقرا: فرض می‌کنیم حکم برای گام زمانی $t - 1$ برقرار باشد. یعنی تا گام قبلی، سلول LSTM زوجیت را به درستی محاسبه و ذخیره کرده است:

$$c_{t-1} = p_{t-1}$$

با توجه به اینکه پیش‌تر اثبات کردیم $h_{t-1} = c_{t-1}$ ، پس $h_{t-1} = p_{t-1}$ نیز برقرار است. اثبات ریاضی با جایگذاری مستقیم پارامترها در معادلات و بررسی حالت‌های ورودی در ادامه آورده شده است.

گام استقرا:

باید ثابت کنیم در گام زمانی t با دریافت ورودی $x_t \in \{0, 1\}$ ، رابطه $c_t = p_t$ نیز برقرار می‌ماند. ابتدا معادلات دروازه‌ها را با جایگذاری مقادیر پارامترهای استخراج‌شده (b و W) بازنویسی می‌کنیم:

$$f_t = \sigma(0 \cdot c_{t-1} - 2 \cdot x_t + 1) = \sigma(1 - 2x_t)$$

$$i_t = \sigma(-4 \cdot c_{t-1} + 3 \cdot x_t - 1)$$

$$\tilde{c}_t = \tanh(0 \cdot c_{t-1} + 1 \cdot x_t + 0) = \tanh(x_t)$$

اکنون دو حالت ممکن برای ورودی جدید (x_t) را بررسی می‌کنیم:

حالت اول: ورودی جدید صفر است ($x_t = 0$)

$$f_t = \sigma(1 - 0) = \sigma(1) = 1$$

$$: i_t = \sigma(-4c_{t-1} - 1)$$

از آنجا که $c_{t-1} \geq 0$ است، عبارت درون سیگموئید همواره منفی است، پس $i_t = 0$.

$$\tilde{c}_t = \tanh(0) = 0$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$c_t = (1 \cdot c_{t-1}) + (0 \cdot 0) = c_{t-1}$$

از نظر منطق زوجیت نیز، با اضافه شدن صفر، زوجیت تغییری نمی‌کند ($p_t = p_{t-1} \oplus 0 = p_{t-1}$). بنابراین $c_t = p_t$ برقرار است.

حالت دوم: ورودی جدید یک است ($x_t = 1$)

مقدار $x_t = 1$ را در معادلات قرار می‌دهیم:

$$f_t = \sigma(1 - 2) = \sigma(-1) = 0$$

$$i_t = \sigma(-4c_{t-1} + 3 - 1) = \sigma(2 - 4c_{t-1})$$

$$\tilde{c}_t = \tanh(1) = 1$$

$$c_t = (0 \cdot c_{t-1}) + (i_t \cdot 1) = i_t$$

پس مقدار c_t در این حالت مستقیماً به مقدار i_t بستگی دارد. دروازه ورودی $i_t = \sigma(2 - 4c_{t-1})$ را بر اساس فرض استقرا ($c_{t-1} \in \{0, 1\}$) در دو زیرحالت بررسی می‌کنیم:

- حافظه قبلی صفر بوده است ($c_{t-1} = 0$)

$$i_t = \sigma(2 - 0) = \sigma(2) = 1 \implies c_t = 1$$

تا الان تعداد یک‌ها زوج بوده ($p_{t-1} = 0$)، با دیدن یک ۱ جدید، باید فرد بشود ($p_t = 1$).

$$نتیجه: c_t = p_t$$

- حافظه قبلی یک بوده است ($c_{t-1} = 1$)

$$i_t = \sigma(2 - 4) = \sigma(-2) = 0 \implies c_t = 0$$

تا الان تعداد یک‌ها فرد بوده $(p_{t-1} = 1)$ ، با دیدن یک ۱ جدید، باید زوج بشود $(p_t = 0)$.

نتیجه: $C_t = p_t$.

۲.۱ پیاده‌سازی

۱.۲.۱ پیش‌پردازش داده‌ها و واژه‌نامه

پیاده‌سازی توکنایزر قانون‌محور فاصله‌گذار

در این مرحله، یک توکنایزر ساده مبتنی بر تفکیک با فاصله (whitespace rule-based tokenizer) پیاده‌سازی شده است. تابع `tokenize(text)` ابتدا کل متن ورودی را با استفاده از `lower()` به حروف کوچک تبدیل می‌کند تا حساسیت به بزرگی و کوچکی حروف از بین برود. سپس، در یک حلقه، تمام کاراکترهای غیر آلفانومریک (مانند علائم نگارشی) فیلتر شده و حذف می‌شوند، به طوری که تنها حروف، اعداد و فواصل باقی بمانند. در نهایت، متن پاک‌سازی شده بر اساس فواصل (whitespace) و به کلمات مجزا (توکن‌ها) تفکیک شده و در قالب یک لیست برگردانده می‌شود. برای ارزیابی عملکرد این تابع، جمله آزمایشی زیر به آن داده شد: "Artin is very tired. Artin has too many DEADLINES!!!! AHHHHH"

خروجی به‌دست‌آمده به شکل زیر است: ['artin', 'is', 'very', 'tired', 'artin', 'has', 'too', 'many', 'deadlines', 'ahhhhh']

همان‌طور که در خروجی مشاهده می‌شود، توکنایزر با موفقیت تمامی علائم نگارشی (مانند . و !) را حذف کرده است. همچنین کلماتی که با حروف بزرگ نوشته شده بودند (مانند DEADLINES و AHHHHH) به‌درستی به حروف کوچک تبدیل شده‌اند تا یکپارچگی داده‌ها حفظ شود. در نهایت، متن دقیقاً بر اساس فواصل موجود به لیستی از توکن‌های استاندارد تجزیه شده است.

مزایای توکنایزر مبتنی بر فاصله:

سادگی و سرعت: این الگوریتم‌ها از نظر محاسباتی بسیار سبک و سریع هستند و پیاده‌سازی آن‌ها به هیچ فرآیند آموزش اولیه‌ای نیاز ندارد.

تطابق با درک انسانی: چون که ما هم خودمون کلمات را براساس فاصله جدا می‌کنیم، خوانایی بالایی برای انسان دارد.

محدودیت‌های جدی و مقایسه با Subword Tokenizers: به صورت کلی می‌شود دو سریک طیف را تصور کرد، در یک طرف هر حرفی را یک توکن در نظر بگیریم که آن موقع تعداد توکن هایمان کم هست ولی بار معنایی را از دست دادیم و در طرف دیگر، هر کلمه یک توکن هست که آن موقع تعداد توکن ها زیاد می‌شود و برای کلماتی مثل `run` و `running` دو توکن کاملاً جدا گرفته می‌شود و حداقل تا این مرحله (بدون آموزش دادن مدل) این دو کلمه کاملاً بی ربط دیده می‌شوند. اما الگوریتم‌هایی وجود دارد که بین این دو طیف را هدف می‌گیرند یعنی هر کلمه را به چندین توکن می‌شکنند مثل Byte-Pair Encoding (BPE) که اول هر حرف را یک توکن نگاه می‌کند و بعد می‌پرسد کدام توکن ها بیشتر کنار هم می‌آید و توکن های جدید می‌سازد.

نحوه مواجهه با کلمات خارج از واژه‌نامه

توکنایزر فاصله‌گذار: اگر مدل در زمان تست با کلمه‌ای مواجه شود که در واژه‌نامه آموزش وجود ندارد، آن را به عنوان یک توکن کاملاً ناشناخته (`<unk>`) در نظر می‌گیرد. این امر باعث از دست رفتن کامل اطلاعات معنایی آن کلمه می‌شود.

توکنایزر زیرکلمه‌ای (Subword): الگوریتم‌هایی مانند BPE در مواجهه با کلمات جدید، آن‌ها را به زیرکلمه‌های شناخته‌شده تجزیه می‌کنند. به عنوان مثال، اگر کلمه "unbelievably" در واژه‌نامه نباشد، ممکن است به "ably" و "believ" و "un" شکسته شود. در نتیجه، مدل همچنان می‌تواند معنای کلمه را بر اساس اجزای سازنده آن درک کند و مشکل کلمه جدید تا حد خوبی حل می‌شود.

چندریختی کلمات:

توکنایزر فاصله‌گذار: کلماتی که ریشه یکسان اما پسوند/پیشوندهای متفاوتی دارند (مانند "play"، "playing"، "played") را به عنوان توکن‌هایی کاملاً مستقل و بی‌ارتباط به یکدیگر در نظر می‌گیرد. این مسئله باعث می‌شود مدل نتواند ارتباط معنایی بین فرم‌های مختلف یک کلمه را به سادگی یاد بگیرد و نیازمند مشاهده مثال‌های فراوان از هر شکل کلمه است.

توکنایزر زیرکلمه‌ای (Subword): این روش‌ها به صورت خودکار ریشه‌های پرکاربرد و پسوندها را شناسایی می‌کنند. بنابراین، ارتباط ساختاری و معنایی بین اشکال مختلف یک کلمه حفظ می‌شود (مثلاً تشخیص "play" به عنوان بخش مشترک).

اندازه بهینه واژه‌نامه:

توکنایزر فاصله‌گذار: برای پوشش دادن حداکثری زبان، نیازمند واژه‌نامه‌ای بسیار بزرگ (گاهی صدها هزار توکن) است تا تمامی اشکال کلمات، اسامی خاص و ترکیبات را در بر بگیرد. این موضوع باعث افزایش شدید حجم ماتریس Embedding و در نتیجه افزایش پارامترهای مدل و نیاز به حافظه بیشتر می‌شود.

توکنایزر زیرکلمه‌ای (Subword): با استفاده از یک واژه‌نامه با اندازه ثابت و بهینه، می‌تواند هر چی کلمه مختلف را با ترکیب زیرکلمه‌ها بازتولید کند. این ویژگی باعث بهینه‌سازی فوق‌العاده در مصرف حافظه و افزایش کارایی مدل‌های زبانی می‌شود.

ساخت واژه‌نامه فرکانسی

توضیح پیاده‌سازی:

در این گام، کلاس TokenToID برای مدیریت واژه‌نامه (Vocabulary) طراحی شده است. این کلاس وظیفه دارد فرکانس حضور هر توکن را در مجموعه داده محاسبه کرده و پرتکرارترین کلمات را برای ساخت فضای Embedding استخراج کند. روند کار به این صورت است که ابتدا توکن‌های ویژه <pad> با شناسه ۰ و <unk> با شناسه ۱ در دیکشنری‌های نگاشت (word2id و id2word) مقداردهی اولیه می‌شوند. با فراخوانی متد add_token، فرکانس هر کلمه محاسبه و در word_freqs ذخیره می‌شود. بعد، متد update_vocab کلمات را بر اساس تعداد تکرار به صورت نزولی مرتب کرده و تنها به اندازه ظرفیت تعیین شده (حداکثر ۵۰۰۰ کلمه پرتکرار)، به آن‌ها شناسه‌های عددی (از ۲ به بعد) اختصاص می‌دهد. در نهایت، متد encode لیستی از توکن‌های متنی را دریافت کرده و با استفاده از دیکشنری ساخته شده، آن‌ها را به دنباله‌ای از اعداد صحیح تبدیل می‌کند.

توضیح منطق و روند:

ابتدا یک گذر کامل روی تمام متون مجموعه آموزش انجام می‌شود تا تعداد دفعات تکرار هر توکن استخراج شود. کلمات بر اساس فرکانس به صورت نزولی مرتب می‌شوند. برای جلوگیری از انفجار ابعاد ماتریس وزن‌ها در لایه Embedding، تنها N کلمه اول (در این پروژه ۵۰۰۰ کلمه) نگه داشته می‌شوند. به توکن‌های باقی‌مانده، شناسه‌های عددی یکتا (از ۲ به بعد) اختصاص می‌یابد. کلماتی که در مجموعه آموزش فرکانس پایینی دارند (خارج از ۵۰۰۰ کلمه برتر)

و همچنین تمام کلمات جدیدی که مدل در مرحله اعتبارسنجی (Validation) یا تست با آن‌ها مواجه می‌شود، در واژه‌نامه دارای شناسه اختصاصی نخواهند بود. در متد encode، یک if-else داریم که اگر توکنی در دیکشنری word2id یافت نشد، مقدار word2id['<unk>'] یعنی عدد ۱ برگردانده شود. این مکانیزم به شبکه عصبی آموزش می‌دهد که چگونه با کلمات ناآشنا برخورد کند. به جای توقف برنامه یا ایجاد خطا، شبکه یاد می‌گیرد که یک Representation کلی برای تمامی کلمات ناشناخته بسازد که معمولاً با تکیه بر اساس context جمله معنای آن‌ها را حدس می‌زند. در پردازش دسته‌ای (Batch Processing) با شبکه‌های عصبی (مانند RNN و LSTM)، تمامی دنباله‌های موجود در یک دسته باید طول یکسانی داشته باشند. از آنجا که جملات طول متفاوتی دارند، جملات کوتاه‌تر باید با توکن خاصی پر شوند تا هم‌طول بلندترین جمله شوند. شناسه ۰ به صورت انحصاری به <pad> اختصاص داده می‌شود. در هنگام تعریف لایه nn.Embedding در PyTorch، می‌توان با تنظیم پارامتر padding_idx=0، از محاسبه گرادیان برای این توکن‌ها جلوگیری کرد. این کار باعث می‌شود تا توکن‌های اضافه‌شده تأثیری در یادگیری و محاسبات وزن‌ها نداشته باشند و صرفاً نقش پرکننده ابعادی را ایفا کنند.

بارگیری فایل‌ها و تقسیم‌بندی داده‌ها

در تابع load_and_split_data، فایل‌های rt-polarity.pos و rt-polarity.neg با استفاده از رمزگذاری latin-1 خوانده می‌شوند. برای اطمینان از توزیع کاملاً متوازن داده‌ها، ابتدا نمونه‌های مثبت و منفی به Shuffle می‌شوند. بعد بر اساس تعداد دقیق خواسته‌شده در صورت پروژه، از هر کلاس (مثبت و منفی) تعداد برابری جدا می‌شود:

مجموعه آموزش (Train): ۴۲۶۵ نمونه مثبت + ۴۲۶۵ نمونه منفی = ۸۵۳۰ نمونه.

مجموعه اعتبارسنجی (Validation): ۵۳۳ نمونه مثبت + ۵۳۳ نمونه منفی = ۱۰۶۶ نمونه.

مجموعه تست (Test): ۵۳۳ نمونه مثبت + ۵۳۳ نمونه منفی = ۱۰۶۶ نمونه.

در نهایت، داده‌های ترکیب‌شده در هر مجموعه مجدداً Shuffle می‌شوند. تا مدل در حین آموزش، نمونه‌های مثبت و منفی را به صورت متوالی و تصادفی دریافت کند و دچار Bias نشود. همچنین در انتهای این بخش، واژه‌نامه نهایی با ظرفیت ۵۰۰۰ کلمه تنها با استفاده از کلمات موجود در مجموعه آموزش ساخته می‌شود. برای بررسی صحت تقسیم‌بندی داده‌ها و اطمینان از همگونی آن‌ها، تابع print_dataset_metrics میانگین و انحراف معیار طول جملات (بر اساس تعداد توکن‌ها) را برای هر سه مجموعه محاسبه کرده است. نتایج به دست آمده به صورت زیر است:

مجموعه آموزش (Train): میانگین طول ۵۱.۱۸ توکن | انحراف معیار ۵۷.۸ | اندازه ۸۵۳۰

مجموعه اعتبارسنجی (Validation): میانگین طول ۴۳.۱۸ توکن | انحراف معیار ۷۴.۸ | اندازه ۱۰۶۶

مجموعه تست (Test): میانگین طول ۴۹.۱۸ توکن | انحراف معیار ۵۷.۸ | اندازه ۱۰۶۶

سایز مجموعه‌ها دقیقاً مطابق با خواسته پروژه (۸۵۳۰ برای آموزش و ۱۰۶۶ برای دو مجموعه دیگر) به دست آمده است. همان‌طور که مشاهده می‌شود، میانگین طول جملات در هر سه مجموعه تقریباً برابر با ۵۰.۱۸ کلمه و پراکندگی (انحراف معیار) آن‌ها نیز در حدود ۶۰.۸ کلمه است. برابری تقریبی این آمارها نشان می‌دهد که فرآیند shuffle تصادفی با seed=42 توزیع یکسانی از طول جملات را در هر سه زیرمجموعه ایجاد کرده است. این خیلی مهم است چون اگر مجموعه تست شامل جملاتی به مراتب طولانی‌تر یا کوتاه‌تر از مجموعه آموزش می‌بود، مدل بازگشتی (LSTM/RNN) در زمان ارزیابی (تست) با داده‌هایی خارج از توزیع آموزشی خود مواجه می‌شد که می‌توانست به افت شدید دقت بینجامد. علاوه بر این، توزیع مشابه طول جملات تضمین می‌کند که استراتژی‌های Padding اعمال شده در فاز آموزش، برای فاز تست نیز کاملاً کارآمد و بهینه خواهند بود.

۲.۲.۱ معماری مدل و بارگذاری داده

پیاده‌سازی تابع پدینگ پویا و آماده‌سازی دسته‌ها

در این بخش، برای تغذیه صحیح داده‌ها به شبکه‌های عصبی بازگشتی، کلاس `SentimentDataset` و تابع `custom_collate_fn` پیاده‌سازی شده‌اند:

کلاس `SentimentDataset`: این کلاس از `torch.utils.data.Dataset` ارث‌بری می‌کند. در متد `__getitem__`، هر متن ابتدا توسط توکنایزر به کلمات مجزا تجزیه شده و سپس با استفاده از واژه‌نامه (`Vocabulary`) ساخته‌شده در مرحله قبل، به دنباله‌ای از شناسه‌های عددی تبدیل می‌شود.

تابع `custom_collate_fn`: شبکه‌های عصبی نیازمند دریافت داده‌ها به صورت دسته‌ای (`Batch`) با ابعاد ماتریسی منظم هستند. از آنجا که طول جملات متفاوت است، این تابع وظیفه هم‌طول کردن آن‌ها را بر عهده دارد. ابتدا طول واقعی هر جمله پیش از پدینگ استخراج و در تانسور `lengths` ذخیره می‌شود (این طول‌ها برای فشرده‌سازی در مدل ضروری هستند). سپس با استفاده از تابع `pad_sequence`، جملات کوتاه‌تر درون همان دسته، با توکن `<pad>` (که ارزش صفر دارد) پر می‌شوند تا هم‌طول بلندترین جمله آن دسته شوند.

ایجاد `DataLoaders`: در نهایت، داده‌های آموزش، اعتبارسنجی و تست در قالب `DataLoader` با اندازه دسته ۶۴ بسته‌بندی شدند.

با اجرای یک حلقه روی `train_loader` و استخراج یک دسته نمونه، ابعاد زیر به دست آمد:

ابعاد `Batch Texts Shape` برابر با `[۴۸، ۶۴]`: عدد ۶۴ نشان‌دهنده اندازه دسته است؛ یعنی دقیقاً ۶۴ نظر فیلم در این تکه از داده‌ها وجود دارد. عدد ۴۸ نشان‌دهنده طول بلندترین نظر در همین دسته خاص است. سایر جملات کوتاه‌تر با صفرهای پدینگ پر شده‌اند تا طولشان به ۴۸ برسد.

ابعاد `Batch Labels Shape` برابر با `[۶۴]`: نشان می‌دهد که دقیقاً ۶۴ برچسب (۰.۱ برای نظرات مثبت و ۰.۰ برای نظرات منفی) متناظر با داده‌های ورودی وجود دارد.

آرایه `Original Sequence Lengths`: این تانسور شامل طول واقعی و اولیه (بدون پدینگ) هر یک از ۶۴ جمله است که در معماری مدل برای نادیده گرفتن مقادیر افزوده شده استفاده خواهد شد. در مدیریت دنباله‌های متنی با طول متغیر، دو رویکرد کلی برای پدینگ وجود دارد: پدینگ ثابت (`Static Global Padding`) و پدینگ پویا در سطح دسته (`Dynamic Batch-level Padding`).

در پدینگ ثابت، تمامی جملات موجود در مجموعه داده تا رسیدن به طول بلندترین جمله کل دیتاست پد می‌شوند. به عنوان مثال، اگر میانگین طول جملات حدود ۱۸ توکن باشد (همان‌طور که در بخش قبل محاسبه شد) اما بلندترین نظر در کل دیتاست ۱۰۰۰ کلمه طول داشته باشد، در پدینگ ثابت همه جملات باید به طول ۱۰۰۰ برسند. این یعنی برای پردازش یک جمله ۱۰ کلمه‌ای، GPU مجبور است ۹۹۰ گام محاسباتی کاملاً بی‌فایده روی توکن‌های صفر انجام دهد که منجر به اتلاف شدید حافظه و کندی چشمگیر آموزش می‌شود.

اما در پدینگ پویا، طول پدینگ تنها محدود به بلندترین جمله در همان دسته فعلی می‌شود (مانند عدد ۴۸ در نمونه خروجی ما). این کار باعث می‌شود طول دنباله‌ها به شدت کوتاه‌تر بماند و از انجام پردازش‌های اضافی و اشغال غیرضروری حافظه GPU جلوگیری شود.

توابعی نظیر `pack_padded_sequence` در `PyTorch` دقیقاً برای تکمیل فرآیند پدینگ پویا طراحی شده‌اند. این تابع، ماتریس پد شده و لیست طول واقعی دنباله‌ها (`lengths`) را به عنوان ورودی دریافت می‌کند.

این تابع ماتریس را از حالت مستطیلی خارج کرده و به یک ساختار فشرده و یک‌بعدی تبدیل می‌کند. در این ساختار، توکن‌های مربوط به هر گام زمانی در کنار هم قرار می‌گیرند، اما توکن‌های پدینگ (صفرها) به طور کامل حذف شده و پردازش نمی‌شوند. سلول بازگشتی (GRU/LSTM/RNN) تنها روی توکن‌های واقعی محاسبات را انجام می‌دهد و در نتیجه سرعت آموزش به‌طور چشمگیری افزایش می‌یابد. اگر صفرهای پدینگ پردازش شوند، مدل در گام‌های زمانی مربوط به پدینگ، Hidden State خود را به‌روزرسانی می‌کند. این امر باعث می‌شود حالت پنهان نهایی دچار تغییرات بی‌مورد شده و اصطلاحاً آلوده شود؛ همچنین محاسبه گرادیان روی این صفرهای بی‌معنی انجام می‌گیرد که می‌تواند منجر به کاهش دقت مدل و اختلال در همگرایی شود. استفاده از `pack_padded_sequence` به طور کامل از این خطای محاسباتی و نظری جلوگیری می‌کند.

طراحی معماری مدل‌های بازگشتی (GRU, LSTM, RNN)

در این مرحله، کلاس چندمنظوره `SentimentRNN` (که از `nn.Module` ارث‌بری می‌کند) به گونه‌ای طراحی شده است که با دریافت پارامتر `rnn_type`، بتواند هر سه معماری `Vanilla RNN`، `LSTM` و `GRU` را `instance` بگیرد.

۱. لایه `nn.Embedding`: در ابتدا، شناسه‌های متنی وارد این لایه می‌شوند. ابعاد فضای `D=64 embedding` در نظر گرفته شده و با تنظیم `padding_idx=0`، به فریم‌ورک اعلام شده است که توکن‌های پدینگ تأثیری در آموزش این لایه نداشته باشند.

۲. لایه بازگشتی: بر اساس نوع معماری انتخابی، یکی از لایه‌های `nn.RNN`، `nn.LSTM` یا `nn.GRU` با ابعاد پنهان `H=128` ساخته می‌شود.

۳. لایه طبقه‌بند (`nn.Linear`): یک لایه `Fully Connected` طراحی شده است تا بردار حالت پنهان نهایی (با طول ۱۲۸) را دریافت کرده و به یک عدد واحد (`Logit`) برای دسته‌بندی باینری (مثبت/منفی) نگاشت کند.

پیش از ورود تنسور `embed` شده به لایه بازگشتی، از تابع `pack_padded_sequence` به همراه طول واقعی جملات (`text_lengths`) استفاده شده است تا شبکه صفرهای پدینگ را پردازش نکند.

نحوه استخراج حالت پنهان آخرین کلمه واقعی دنباله:

در کتابخانه `PyTorch`، زمانی که از `pack_padded_sequence` برای فشرده‌سازی ورودی‌ها استفاده می‌کنیم، خروجی حالت پنهان (`hidden`) که توسط ماژول‌های `LSTM/GRU/RNN` برگردانده می‌شود، به‌صورت خودکار معادل با حالت پنهان آخرین کلمه واقعی (قبل از شروع توکن‌های پدینگ) برای هر جمله درون دسته است. بنابراین، نیازی به اندیس‌گذاری دستی روی خروجی‌ها نداریم. ما این تنسور `hidden` را دریافت کرده، با استفاده از `squeeze(0)` بعد اضافی مربوط به تعداد لایه‌ها (که در اینجا ۱ است) را حذف می‌کنیم و مستقیماً به لایه خطی می‌دهیم.

تفاوت در خروجی مدل LSTM:

در `Vanilla RNN` و `GRU` تنها یک ساختار حافظه واحد به نام حالت پنهان (`Hidden State`) دارند و در نتیجه خروجی متد `forward` آن‌ها شامل دو المان (`hidden`، `packed_output`) است. اما معماری `LSTM` برای حل مشکل ناپدید شدن گرادیان، دارای یک مسیر انتقال اطلاعات اضافه به نام حالت سلول (`Cell State`) است. بنابراین، متد `forward` در `nn.LSTM` یک تاپل متشکل از خروجی کل، و زوج حالت‌ها را برمی‌گرداند: (`packed_output`، (`hidden`، `cell`)). با استفاده از یک شرط `if self.rnn_type == 'lstm'`، ما این زوج را دریافت کرده و متغیر `cell` را نادیده می‌گیریم، زیرا برای طبقه‌بندی نهایی تنها به `hidden` (که نمایانگر ویژگی‌های استخراج شده در آخرین گام است) نیاز داریم.

اگر از `pack_padded_sequence` استفاده نمی‌کردیم و تنسورهای پد شده را مستقیماً به شبکه می‌دادیم، برای استخراج خروجی نهایی مجبور بودیم آخرین گام زمانی خروجی را برداریم (مثلاً با اندیس‌گذاری `output[:, -1, :]`). این رویکرد یک خطای منطقی شدید است؛ زیرا برای جملاتی که کوتاه‌تر از ماکزیمم طول دسته هستند، آخرین گام زمانی در واقع مربوط به پردازش توکن‌های `<pad>` است، نه کلمات واقعی جمله.

وقتی شبکه کلمات صفر (پدینگ) را پردازش می‌کند، بردار حالت پنهانی که در کلمات واقعی جمله ساخته شده بود، با ضرب شدن در ماتریس‌های وزن و عبور از توابع فعال‌ساز ($\tanh$) در گام‌های پدینگ، به تدریج تحلیل رفته، تغییر شکل داده و اطلاعات مفید آن از بین می‌رود. در نتیجه، برداری که به لایه دسته‌بند می‌رسد، نماینده معنای جمله نیست، بلکه نماینده پردازش تعدادی توکن بی‌معنی است. این امر به شدت دقت شبکه را کاهش داده و فرآیند همگرایی را مختل می‌کند.

۳.۲.۱ منحنی‌های یادگیری و مقایسه عملکرد

آموزش مدل‌ها و استخراج آمارها در این بخش، توابع لازم برای آموزش و ارزیابی مدل‌ها و در نهایت حلقه اصلی آموزش پیاده‌سازی شده‌اند. تابع `binary_accuracy` با اعمال تابع فعال‌ساز `sigmoid` بر روی خروجی‌های خام مدل (`Logits`) و گرد کردن آن‌ها به ۰ یا ۱، دقت پیش‌بینی‌ها را محاسبه می‌کند. توابع `train_epoch` و `evaluate` وظیفه‌گذارند داده‌ها از مدل، محاسبه خطای باینری (`BCEWithLogitsLoss`) و به‌روزرسانی وزن‌ها از طریق الگوریتم بهینه‌ساز `Adam` (فقط در مرحله آموزش) را بر عهده دارند. هر سه مدل (`LSTM`، `GRU`، `RNN`) برای ۵ اپیاک آموزش دیدند. در هر گام، تعداد پارامترهای قابل آموزش محاسبه شده و زمان اجرای هر اپیاک با استفاده از کتابخانه `time` اندازه‌گیری و تجمیع می‌شود تا زمان کل آموزش به دست آید. تاریخچه خطای آموزش و اعتبارسنجی (`Loss`) و همچنین دقت (`Accuracy`) برای رسم نمودار در دیکشنری `history` ذخیره شد. بر اساس خروجی‌های چاپ‌شده در پایان فرآیند آموزش، جدول آماری مقایسه سه معماری به شرح زیر است:

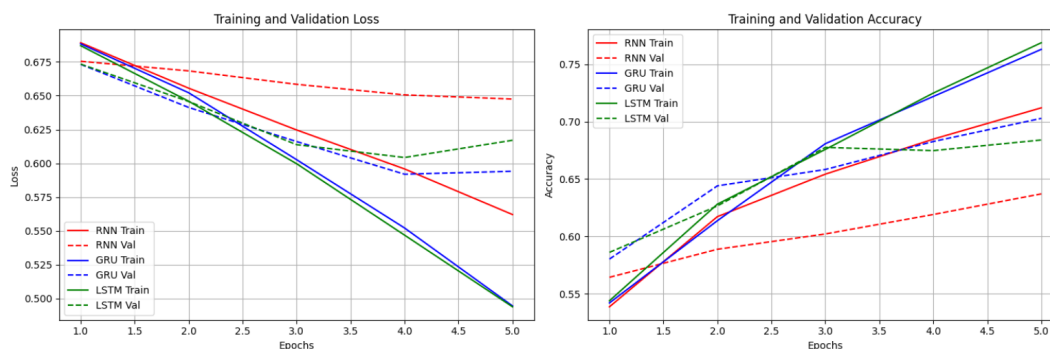
مدل معماری	تعداد پارامترهای قابل آموزش	زمان کل آموزش (ثانیه)	دقت اعتبارسنجی	دقت تست
Vanilla RNN	344961	12.89	63.71%	65.86%
GRU	394625	25.79	70.29%	70.10%
LSTM	419457	30.76	68.41%	66.00%

در معماری `Vanilla RNN`، سلول بازگشتی در هر گام زمانی تنها یک تبدیل خطی ساده (ضرب ماتریسی بردار ورودی و حالت پنهان قبلی در ماتریس وزن‌ها) و گذراندن آن از یک تابع فعال‌ساز را انجام می‌دهد. به همین دلیل پردازش آن بسیار سبک بوده و زمان آموزش در اینجا تنها حدود ۱۳ ثانیه است (بیش از ۲ برابر سریع‌تر از بقیه). این مدل از پدیده ناپدید شدن گرادیان رنج می‌برد. به همین دلیل، در جملات طولانی نظرات فیلم، نمی‌تواند وابستگی‌های بلندمدت کلمات را به خوبی یاد بگیرد. ناتوانی در حفظ اطلاعات دور، منجر به استخراج ویژگی‌های ضعیف‌تر و در نتیجه دقت پایین‌تر (حدود ۶۵٪) نسبت به مدل‌های دروازه‌دار می‌شود. در مقابل، `LSTM` و `GRU` با استفاده از مکانیزم‌های دروازه‌ای محاسبات ریاضی بسیار سنگین‌تری در هر گام دارند (که دلیل کندی آن‌هاست)، اما این مکانیزم‌ها جریان گرادیان را حفظ کرده و با درک بهتر جملات، به دقت‌های بالاتری می‌رسند (البته بحث `overfitting` نیز هست که بر دقت تست تاثیر می‌گذارد که جلوتر توضیح دادم).

رسم منحنی‌های آموزش و اعتبارسنجی در ادامه، نمودارهای دوتایی همجوار که نشان‌دهنده روند تغییرات خطا و دقت روی داده‌های آموزش و اعتبارسنجی هستند، آورده شده است:

تحلیل وضعیت بیش‌برازش (`Overfitting`) در مدل‌ها: با بررسی دقیق منحنی‌های ثبت شده (به ویژه پل سمت چپ مربوط به `Loss`)، می‌توان رفتار تعمیم‌پذیری هر سه مدل را به صورت زیر ارزیابی کرد:

مدل `Vanilla RNN`: این مدل دچار بیش‌برازش شدید نشده است. خطای اعتبارسنجی آن با شیبی ملایم در حال کاهش است. مشکل اصلی این مدل،



شکل ۱.۱: روند تغییرات خطا و دقت آموزش و اعتبارسنجی مدل‌ها

ظرفیت پایین آن در یادگیری است؛ به طوری که حتی خطای آموزش آن نیز نتوانسته به اندازه دو مدل دیگر کاهش یابد.

مدل GRU: این مدل رفتار همگرایی بسیار خوبی از خود نشان داده است. خطای آموزش با شیب تند کاهش یافته و خطای اعتبارسنجی نیز تا اپیک ۴ روند نزولی دارد. در اپیک ۵، افت نامحسوسی در خطای اعتبارسنجی (از 0.594 به 0.592) دیده می‌شود که نشان‌دهنده شروع بسیار ملایم بیش‌برازش در گام‌های نهایی است. اما در مجموع بهترین تعمیم‌پذیری و بالاترین دقت تست را داشته است.

مدل LSTM: بیش‌برازش در این مدل کاملاً واضح است. مطابق نمودار خطای سمت چپ (خط‌چین سبز رنگ)، خطای اعتبارسنجی در اپیک ۴ کمترین مقدار خود (0.604) می‌رسد و پس از آن در اپیک ۵ شروع به افزایش می‌کند (به 0.617 می‌رسد)، در حالی که خطای آموزش همچنان با قدرت در حال کاهش است. فاصله افتادن بین منحنی خطای آموزش و اعتبارسنجی، نشانه بیش‌برازش است.

همان‌طور که مشاهده شد، مدل LSTM هم زودتر و هم بیشتر دچار بیش‌برازش شده است. دلیل این رفتار، ظرفیت بالای یادگیری این معماری است. مدل LSTM با داشتن سه دروازه (Forget, Input, Output) و یک حالت سلول (Cell State) مجزا، بیشترین تعداد پارامترهای آموزش‌پذیر (۴۱۹,۴۵۷ پارامتر) را در بین این سه مدل دارد. در مقابل دیتاست فعلی که اندازه نسبتاً کوچکی دارد (تنها ۸۵۳۰ نمونه آموزشی)، این حجم از پارامترها و ظرفیت محاسباتی بالا باعث می‌شود مدل به جای کشف الگوهای معنایی عمومی، شروع به حفظ کردن نویزها و جزئیات خاص مجموعه آموزش کند. مدل GRU به دلیل تجمع دروازه‌ها و نداشتن مسیر مجزای حالت سلول، پارامترهای کمتری (۳۹۴,۶۲۵ پارامتر) نسبت به LSTM دارد و به عنوان یک مدل با پیچیدگی نسبی، توانسته تعادل بهتری بین یادگیری الگوها و جلوگیری از بیش‌برازش روی این مجموعه داده برقرار کند.

۴.۲.۱ تشخیص رفتار گرادیان نسبت به گام‌های زمانی

استخراج گرادیان فضای تعبیه‌ساز در طول زمان در این مرحله، تابع `extract_gradients` برای ترک جریان گرادیان طراحی شده است. ابتدا مدل در حالت ارزیابی (`model.eval()`) قرار می‌گیرد. سپس یک دسته با اندازه ۳۲ شامل نمونه‌هایی که حداقل ۳۰ کلمه دارند استخراج شده و طول همه آن‌ها دقیقاً روی ۳۰ توکن اول ($T = 30$) برش می‌خورد. برای ثبت جریان خطا، بردار خروجی لایه تعبیه‌ساز (`embedded`) از گراف محاسباتی قبلی جدا شده (`detach`) و `requires_grad_(True)` برای آن فعال می‌شود. پس از عبور این بردارها از لایه بازگشتی و محاسبه خطای باینری (`BCEWithLogitsLoss`)، تابع `loss.backward()` فراخوانی می‌گردد. در نهایت، با استفاده از `embedded.grad`، مشتق جزئی خطا نسبت به بردارهای ورودی در هر گام زمانی (g_t) استخراج شده و میانگین نرم ۲ (`L2 Norm`) این گرادیان‌ها در طول دسته محاسبه می‌شود.

چرا محاسبه گرادیان نسبت به فضای Embedding؟

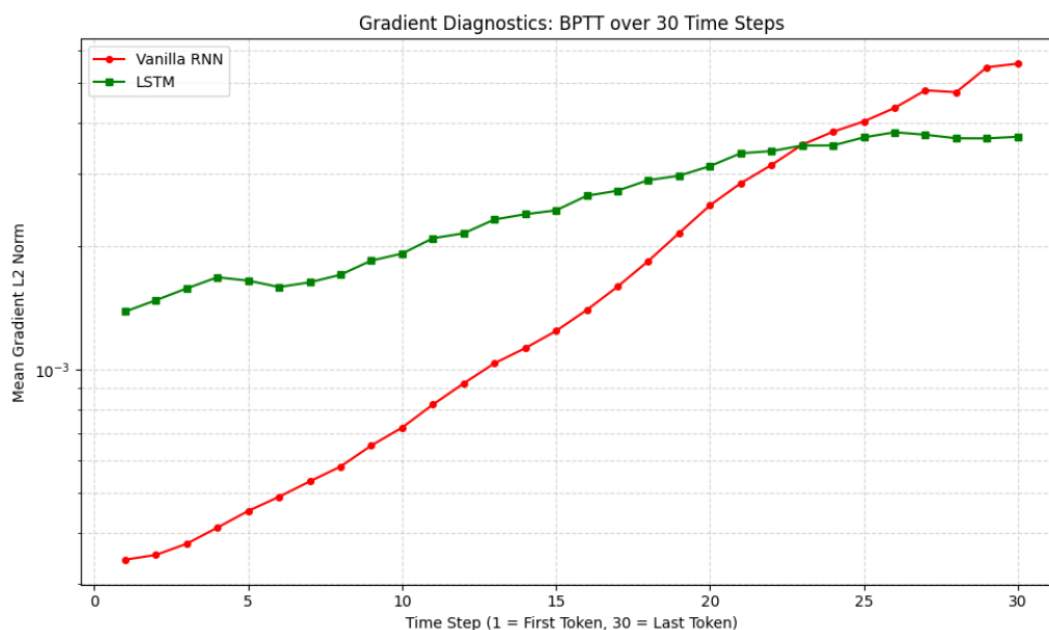
هدف اصلی ما بررسی نحوه انتقال اطلاعات از گام زمانی نهایی ($T = 30$) به گام‌های اولیه (t) است. طبق قاعده زنجیره‌ای مشتق، گرادیان خطا نسبت به ورودی در گام t (یعنی e_t) به صورت زیر محاسبه می‌شود:

$$g_t = \frac{\partial L}{\partial e_t} = \frac{\partial L}{\partial h_T} \frac{\partial h_T}{\partial h_t} \frac{\partial h_t}{\partial e_t}$$

عبارت میانی، یعنی $\frac{\partial h_T}{\partial h_t}$ ، دقیقاً همان ماتریس ژاکوبین زمانی است که پدیده ناپدید یا انفجار گرادیان را رقم می‌زند. از آنجا که دو عبارت دیگر ($\frac{\partial h_t}{\partial e_t}$ و $\frac{\partial L}{\partial h_T}$) مقادیری محلی و محدود هستند، تغییرات g_t به صورت خالص نمایانگر رفتار $\frac{\partial h_T}{\partial h_t}$ است. علاوه بر این باید یک مقایسه عادلانه ما انجام میدادیم. h_t در Vanilla RNN معنای متفاوتی نسبت به h_t در LSTM دارد (چون LSTM حالت سلول هم دارد و کلاً معنای حالتش تغییر کرده). اما اصلاً قبل از اینکه وارد این بحث بشویم، هر دو شبکه (با هر پیچیدگی‌ای که دارند)، دقیقاً یک ورودی مشترک ۶۴ بعدی رو از لایه Embedding می‌گیریم. پس با اندازه‌گیری گرادیان روی Embedding، ما یک نقطه مرزی و کاملاً یکسان برای هر دو مدل داریم و درون مدل را black box انگار در نظر می‌گیریم. به طور خلاصه، وقتی می‌گیریم شبکه قراره وابستگی‌های بلندمدت را یاد بگیره، منظورمون اینه که شبکه بفهمه کلمه اول روی کلمه آخر تأثیر داره. فضای Embedding نماینده مستقیم همون کلمه است. وقتی ما گرادیان رو روی بردار Embedding کلمه اول محاسبه می‌کنیم، داریم می‌گیریم برای کاهش خطای نهایی آیا نیازه معنای کلمه در گام اول را تغییر بدی یا نه پس خیلی منطقی هست بنابر دلایل بالا که ما گرادیان را روی Embedding محاسبه کنیم.

محاسبه نرم گرادیان و رسم نمودار جریان

برای درک بهتر، نمودار لگاریتمی جریان میانگین نرم گرادیان‌ها در طول ۳۰ گام زمانی رسم شده است:



شکل ۲.۱: نمودار لگاریتمی جریان میانگین نرم گرادیان‌ها در طول گام‌های زمانی

در معماری Vanilla RNN، حالت پنهان با رابطه $h_k = \tanh(W_{hh}h_{k-1} + W_{xh}x_k + b)$ به‌روزرسانی می‌شود. در فرآیند BPTT، گرادیان

زمانی باید از حاصل ضرب ژاکوبین‌های متوالی عبور کند:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=t+1}^T \text{diag}(1 - h_k^2) W_{hh}$$

با توجه به اینکه مشتق تابع $\tanh$ همواره کوچکتر یا مساوی ۱ است، اگر بزرگترین مقدار تکین ماتریس وزن W_{hh} نیز از ۱ کوچکتر باشد، نرم هر یک از جملات این حاصل‌ضرب کسری خواهد بود. ضرب متوالی ده‌ها عدد کسری (مربوط به گام‌های زمانی)، منجر به کاهش نمایی مقدار نهایی می‌شود. این را به وضوح در نمودار تجربی بالا (منحنی قرمز رنگ Vanilla RNN) نمایان است. از آنجا که محور y مقیاس لگاریتمی (Log-scale) دارد، یک خط راست با شیب مثبت نشان‌دهنده یک رابطه نمایی است. گرادیان از گام ۳۰ (نزدیک به خروجی) به سمت گام ۱ (اولین کلمه) به صورت نمایی در حال سقوط آزاد است و عملاً در گام‌های ابتدایی به مقادیر بسیار ناچیزی می‌رسد که توانایی آپدیت وزن‌ها را از دست می‌دهد. مدل LSTM برای مقابله با این فروپاشی، مسیر انتقال مستقیمی به نام حالت سلول ایجاد کرده است. معادله به‌روزرسانی در این مسیر به شکل زیر است:

$$c_k = f_k \odot c_{k-1} + i_k \odot \tilde{c}_k$$

هنگام محاسبه مشتق جزئی برای این مسیر در فرآیند BPTT، داریم:

$$\frac{\partial c_k}{\partial c_{k-1}} \approx \text{diag}(f_k) + \dots$$

اگر شبکه یاد بگیرد که گیت فراموشی را برای حفظ اطلاعات بلندمدت کاملاً باز نگه دارد (یعنی $f_k \approx 1$)، مشتق حالت فعلی نسبت به حالت قبل تقریباً برابر با ماتریس همانی خواهد شد. در نتیجه:

$$\frac{\partial c_T}{\partial c_t} \approx \prod_{k=t+1}^T 1 = 1$$

این مکانیزم (Constant Error Carousel - CEC)، باعث می‌شود گرادیان بدون ضرب شدن در ضرایب کاهنده، در طول زمان به عقب برگردد. در نمودار تجربی (منحنی سبز رنگ LSTM)، این ثبات کاملاً مشهود است؛ شیب نمودار LSTM در مقایسه با RNN بسیار ملایم‌تر بوده و از گام ۳۰ تا گام ۱، افت شدیدی را تجربه نکرده است. بر اساس خروجی‌های به‌دست‌آمده، نسبت نرم گرادیان در گام اول (دورترین کلمه از خروجی) به گام سی‌ام (نزدیک‌ترین کلمه به خروجی) محاسبه شده است:

نسبت حفظ گرادیان در Vanilla RNN: برابر با 6.17×10^{-2}

نسبت حفظ گرادیان در LSTM: برابر با 3.75×10^{-1}

این اعداد نشان می‌دهند که در شبکه Vanilla RNN، قدرت سیگنال یادگیری (گرادیان) وقتی از کلمه سی‌ام به کلمه اول می‌رسد، تقریباً به ۰.۰۶ مقدار اولیه خود افت می‌کند. این یعنی شبکه عملاً به کلمات ابتدایی یک جمله بلند کور شده و نمی‌تواند وابستگی معنایی کلمه اول با کلمه آخر را درک کند. در نقطه مقابل، این نسبت در LSTM برابر با ۰.۳۷۵ است. به عبارت دیگر، LSTM توانسته سیگنال خطای خود را بیش از ۶ برابر قوی‌تر از RNN تا گام اول حفظ کند. این حفظ جریان گرادیان، عامل کلیدی موفقیت شبکه‌های دروازه‌دار در یادگیری الگوهای پیچیده و وابستگی‌های بلندمدت در پردازش زبان طبیعی است.